



© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2023

M. Frisbie, *Building Browser Extensions*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-8725-5_6

6. Understanding the Implications of Manifest V3

Matt Frisbie¹

(1) California, CA, USA

The world of browser extensions is undergoing a major transformation. Google Chrome is spearheading a transition that changes how extensions work and what they are capable of doing, and the rest of the major browsers are likely to follow suit. Some of these changes are controversial, as they have significant implications for many very popular chrome extensions. The changes are embodied within a refashioning in how the `manifest.json` is written (by defining a new `manifest_version` 3); thus, these changes are commonly referred to as “manifest v3.”

Note The transition is far from over, and many details of where the ecosystem will end up remain to be finalized. This chapter will mostly cover information and examples that are certain.

Motivation for Manifest V3

All the major changes contained in manifest v3 can be grouped into a handful of motivations: security, performance, privacy and transparency, and revenue.

Security

Manifest v2 allowed extensions to execute JavaScript that was loaded from remote URLs or provided by the user. This was judged to be problematic, as a malicious script executing with access to extension APIs and permissions could cause a lot of damage. To address this security hole, manifest v3 restricts extensions to only be able to execute scripts which are included in the extension package itself.

Performance

Several manifest v2 features had the potential to introduce performance problems in the browser.

Migration to DeclarativeNetRequest

The manifest v2 `webRequest` API allowed developers to run JavaScript at different points in the lifecycle of a network request. When used in the extreme, extensions that managed *all* network traffic on *all* pages would therefore execute blocking JavaScript for every single network request that the browser makes.

Manifest v3 addresses this by moving the blocking scripts into a static set of declarative rules for how to manage network traffic (block, redirect, etc.). The browser loads these static rules and natively executes them as needed, thereby eliminating the need to run extra JavaScript for each network request.

Migration to Service Workers

Background scripts in manifest v2 were capable of being “persistent,” meaning the script would never terminate. The implication of this was that each extension with a persistent background script would introduce an extra JavaScript runtime that the browser must manage.

Manifest v3 addresses this by migrating background scripts to run as service workers. These service workers will automatically spin up on

demand in response to observed browser events and terminate when there is no activity to free up system resources.

Note The removal of persistent background scripts is highly controversial and causes problems for extensions which require the use of long-lived entities such as websockets. Read on in this chapter for strategies to address these issues.

Privacy and Transparency

Like mobile apps, browser extensions require permission from users before they are allowed to perform all but the most basic of tasks. Many permissions, once granted, provide unfiltered access to sensitive information such as page content, cookies, and browsing activity. Manifest v3 alters the manifest permissions structure and introduces increased control and visibility into which permissions an extension is using.

Revenue

It is no secret that many tech giants rely heavily on revenue from web advertisements. Some of the most popular browser extensions are “ad blockers,” extensions which are exceptionally adept at preventing users from seeing ads and eliminating any potential for ad revenue. Despite the dominance of mobile web traffic, desktop web traffic continues to be a significant slice of all web traffic – and a large percentage of this slice uses ad blockers.

The companies that lose this ad revenue to ad blockers are the same ones that support the browser extension APIs and browser extension marketplaces that power these ad blockers. To resolve this conflict, manifest v3 takes aim at the extension APIs that ad blockers rely upon. The changes in manifest v3 don’t entirely wipe out the ability to block ads, but it is significantly diminished.

Implications of Background Service Workers

In manifest v2, background scripts could be defined as either a *persistent script* or an *event page*. The persistent script was initialized and kept alive for the entire time that the browser remained open. This allowed the background script to run in parallel with the page, perform work in the top level of the script, open long-lived network connections like websockets, listen for incoming events from external sources, and perform tasks at short intervals, all without the threat of the script being terminated. Extensions that did not have a need for this persistent script state could elect to instead run their background script as an event page. The browser would run the background script to initialize listeners for browser events and then suspend the background script when it was deemed to be idle. When a browser event with a handler was fired, the event page would wake up and run the handler.

Manifest v3 discards the duality of persistent scripts and event pages in favor of *service workers*. In many ways, the background service worker is analogous to an event page. Even so, background scripts in manifest v2 and manifest v3 have several crucial differences:

DOM

In manifest v2, background scripts were created as a headless browser page, including full access to a DOM. In manifest v3, the service worker cannot access the DOM or any DOM APIs. However, it still has access to the OffscreenCanvas API.

XMLHttpRequest

In manifest v2, background scripts were able to make network requests using `XMLHttpRequest`. In manifest v3, because all background scripts are service workers, network requests must be made via `fetch()`.

Timer API

In manifest v2, a persistent script could use a timer API method like `setTimeout()` or `setInterval()` in the top level of the script and have the handler reliably execute. The following is an example of a timeout handler which would reliably run in a manifest v2 extension with a persistent background:

Timer which will reliably run in a persistent background script in manifest v2

```
// Log message after 5 minute delay
setTimeout(() => console.log("5 minutes is up!"), 5 * 60 * 1000);
```

In manifest v3, because there is no option to require persistence, these handlers may not reliably execute. The browser does not factor in these timer handlers into its decision about if a service worker is idle. When the browser decides to stop an idle service worker, these timer handlers will be silently canceled. In the above example, the log statement would never print if the service worker was stopped prior to the handler execution.

The recommended substitute is to use the extension Alarms API. These methods are similar to the timer API methods in that they can fire timed events, but these events *will* wake the service worker and execute the handler. The following example reworks the above example so that it will function properly in a manifest v3 service worker:

Alarm which will reliably run in a background service worker in manifest v3

```
// Schedule an alarm event to be fired in 5 minutes
chrome.alarms.create({ delayInMinutes: 5 });
// Set a handler for alarm events
chrome.alarms.onAlarm.addListener(() => console.log("5 minutes is up!"));
```

In this reworked example, the service worker will wake up for the alarm event. Therefore, it is guaranteed that the handler will run as expected. When comparing these two strategies, you will notice that the Alarms API is a poor substitute for higher frequency events.

Note The Alarms API is covered in the *Extension and Browser APIs* chapter. Refer to the *Background Scripts* chapter for strategies to address high-frequency timers.

Event Handlers

Because service workers are expected to start and stop on a regular basis, the background script must be organized in a particular fashion to ensure correct behavior. Keep the following behavior in mind when writing your service worker:

- Service worker listeners are always terminated when the service worker stops.
- Service worker listeners are always added when the service worker starts.
- When a service worker starts in response to an event, that event will be dispatched to the service worker immediately after it starts.

When writing a background service worker, you should assume that an event may be fired immediately after you set the handler. As a result, event handlers must be registered *in the first turn of the event loop*. The following is an example of a handler which is not registered in the first turn of the event loop, and therefore may miss click events dispatched when the service worker restarts:

Event handler registration which may not handle all events correctly

```
// Bad
setTimeout(
  () => chrome.action.onClicked.addListener(() => console.log("click")),
  10
```

```
);
```

The following is an example of a handler which is registered in the first turn of the event loop, and therefore will correctly handle events dispatched when the service worker restarts:

Event handler registration which will handle all events correctly

```
// Good
chrome.action.onClicked.addListener(
  () => console.log("click"));
```

Tip The rule of thumb is to always register events at the top level of the background script.

Service Worker Persistence

In manifest v2, a persistent background script would be initialized upon installation or browser startup and run uninterrupted until the browser was closed or the extension was uninstalled. In manifest v3, service workers are stopped when the browser detects they are idle.

Consider the following extension, which sets a timer in the background service worker to measure how long it stays alive:

background.js which logs to console until the service worker is shut down

```
const t0 = performance.now();
setInterval(() => {
  const t1 = performance.now();
  console.log(`Alive for ${Math.round((t1 - t0) / 1e3)}s`);
}, 1e3);
```

Tip In Google Chrome, if you visit `chrome://serviceworker-internals/?devtools` you will find the Service Worker Internals interface, which allows you to monitor the real-time status of your service

workers as well as their console output. Unlike the browser inspector interface, which may prevent the browser from recognizing a service worker as idle, this tool will allow the service workers to go idle and be stopped.

The following screenshots show this extension service worker in the RUNNING and STOPPED states (Figures 6-1 and 6-2):

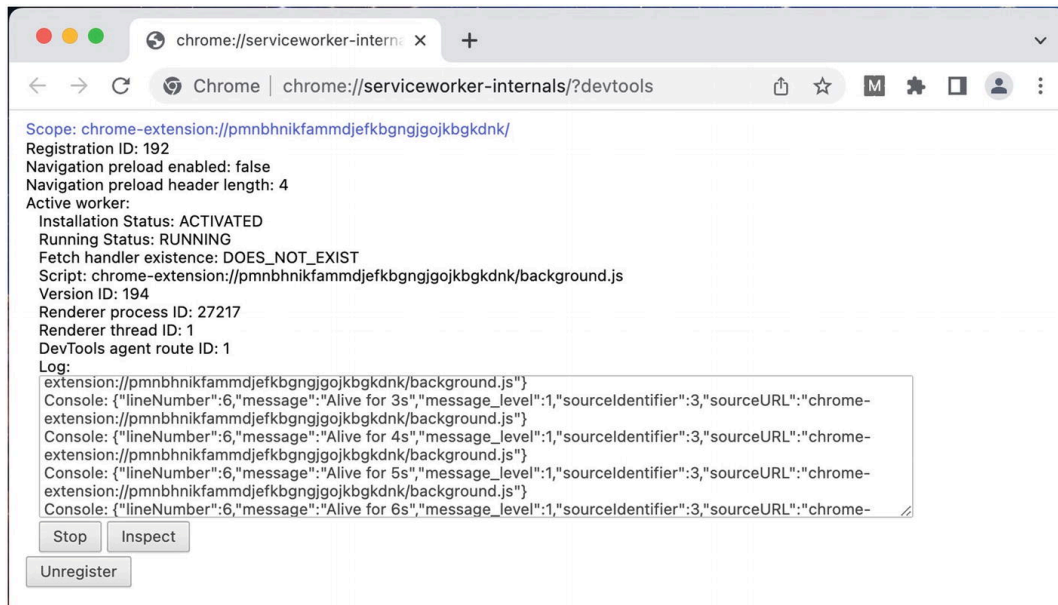


Figure 6-1 The background service worker in the RUNNING state

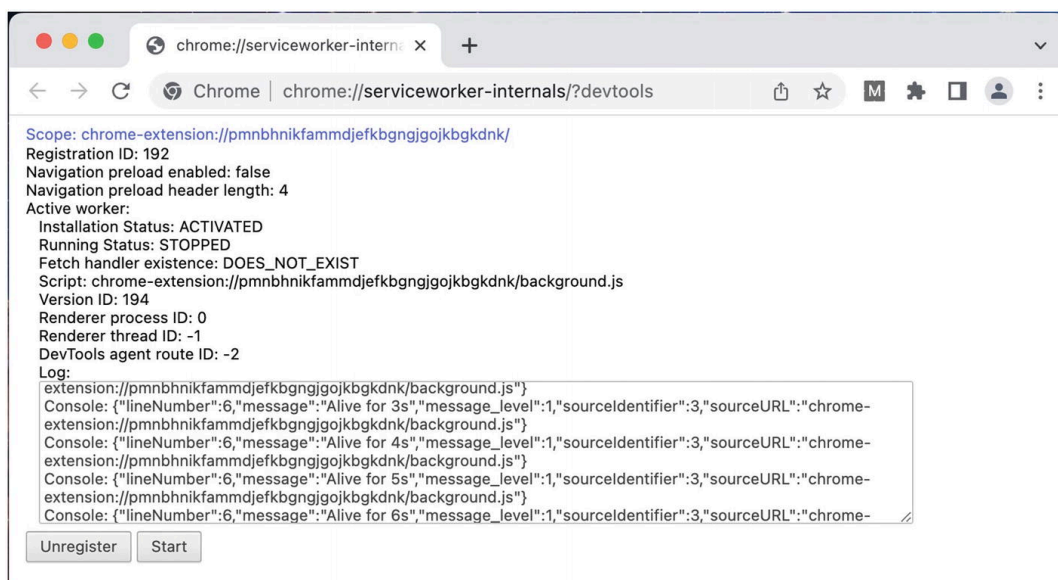


Figure 6-2 The background service worker in the STOPPED state

Even with the interval timer, the browser will consider this script idle. Typically, you will see the timer log for 30 seconds before the browser

stops the service worker.

Open long-lived connections like extension messaging ports or websockets will delay the shutdown of the service worker, but Chrome will still stop the service worker after 5 minutes and sever the connections. Therefore, manifest v3 extensions should be organized as to avoid relying on long-lived connections from the background script.

Note This particular aspect of service workers is extremely controversial. In many cases, long-lived connections in the background script are required for the extension to work properly. There are a handful of tricks to force the service worker to stay running. For more details, refer to the *Background Scripts* chapter.

Global State and Storage

Manifest v2 extensions commonly relied on global state in the background script when using persistent scripts. The following is an example of a simple background script that counts action button clicks:

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 2,
  "background": {
    "scripts": ["background.js"],
    "persistent": true
  },
  "browser_action": {}
}
```

Example 6-1a manifest.json for manifest v2 example

```
let count = 0;
chrome.browserAction.onClicked.addListener(() => {
  console.log(`Clicked ${++count} times`);
});
```

Example 6-1b background.js for manifest v2 example

Because the `count` variable is in a persistent global scope, the extension can access it in perpetuity with no problems. However, consider this example when it is converted to manifest v3:

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "background": {
    "service_worker": "background.js"
  },
  "action": {}
}
```

Example 6-2a manifest.json for manifest v3 example

```
let count = 0;
chrome.action.onClicked.addListener(() => {
  console.log(`Clicked ${++count} times`);
});
```

Example 6-2b background.js for manifest v3 example

This will work correctly until the service worker stops. At that time, the global variable will reset, as any variables in the global scope are lost. The recommended solution is to use the Storage API, which will persist through service worker stops and starts. The following code shows this example refactored to use this API:

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "background": {
    "service_worker": "background.js"
  },
  "action": {},
  "permissions": ["storage"]
}
```

```
}
```

Example 6-3a Refactored manifest.json for manifest v3 example

```
chrome.action.onClicked.addListener(() => {  
  chrome.storage.local.get(["count"], ({ count = 0 }) => {  
    console.log(`Clicked ${++count} times`);  
    chrome.storage.local.set({ count });  
  });  
});
```

Example 6-3b Refactored background.js for manifest v3 example

Note For more details on the Storage API, refer to the *Extension and Browser APIs* chapter.

Audio and Video

Service workers are not able to play or capture media streams in the browser. To use these media APIs, a chrome extension page or a content script must be used.

Implications of Content Security Policy Restrictions

In manifest v2, extensions were allowed to load scripts from remote origins, run user-provided scripts, and run scripts inline in the browser. In manifest v3, these are explicitly forbidden. It is still possible to make use of these in a sandboxed page, but these sandboxed scripts cannot access extension APIs.

The following popup page shows three instances of scripts that will throw runtime errors when loaded in a manifest v3 extension:

popup.js with forbidden eval()

```
// eval() is not allowed
```

```
eval(`console.log('foobar');`);
```

popup.html with forbidden scripts

```
<!DOCTYPE html>
<html>
  <body>
    <h1>Popup</h1>
    <!-- Inline scripts not allowed -->
    <script>
      console.log("foobar");
    </script>
    <script src="popup.js"></script>
    <!-- Loading from remote origin not allowed -->
    <script
      src="https://code.jquery.com/jquery-3.6.0.js"
      integrity="sha256-H+K7U5CnXl1h5ywQfKtSj8PCmoN9aaq30gDh27Xc0jk="
      crossorigin="anonymous"
    ></script>
  </body>
</html>
```

For manifest v3 extensions, the principle is simple: all JavaScript that runs in the extension must be loaded along with the extension. If your extension needs to use third-party libraries, you must include static copies of those libraries in your extension payload. Developers that build their extensions with tools such as Webpack will not have to worry about doing this, as the libraries are already bundled and included.

Note With all these script techniques disabled, in manifest v3 it is no longer possible for extension users to supply their own scripts at run-time to run in any extension. As a result, this change breaks all user-script extensions like Greasemonkey and Tampermonkey.

Implications of DeclarativeNetRequest

The change from `webRequest` to `declarativeNetRequest` isn't strictly part of manifest v3, but the transition is occurring simultaneously, and its impact is just as significant. With `webRequest`, the browser intercepts and routes network traffic to the extension, and the extension programmatically manipulates each request in JavaScript. The `DeclarativeNetRequest` API inverts this paradigm: the extension defines a set of rules to instruct the browser on how to handle each network request, and the browser performs the request manipulation.

For example, the following manifest v2 extension uses a simple URL matcher to block the Google logo from loading:

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 2,
  "background": {
    "scripts": ["background.js"]
  },
  "permissions": [
    "webRequest",
    "webRequestBlocking",
    "<all_urls>"
  ]
}
```

Example 6-4a manifest.json for manifest v2 google logo blocker

```
chrome.webRequest.onBeforeRequest.addListener(
  () => {
    return { cancel: true };
  },
  { urls: ["*://*.google.com/logos/*"] },
  ["blocking"]
);
```

Example 6-4b background.js for manifest v2 google logo blocker

This extension will kill all requests for network requests matching

`*://*.google.com/logos/*`. Load the extension and visit `google.com` to test.

This behavior can be replicated in manifest v3 as follows:

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "permissions": ["declarativeNetRequest"],
  "host_permissions": ["<all_urls>"],
  "declarative_net_request": {
    "rule_resources": [
      {
        "id": "ruleset_1",
        "enabled": true,
        "path": "rules.json"
      }
    ]
  }
}
```

Example 6-5a manifest.json for manifest v3 Google logo blocker

```
[
  {
    "id": 1,
    "priority": 1,
    "action": { "type": "block" },
    "condition": {
      "urlFilter": "*.google.com/logos/*",
      "resourceTypes": ["image"]
    }
  }
]
```

Example 6-5b rules.json for manifest v3 Google logo blocker

Note Details on networking can be found in the *Networking* chapter.

Load google.com again to test the logo blocker (Figure 6-3).

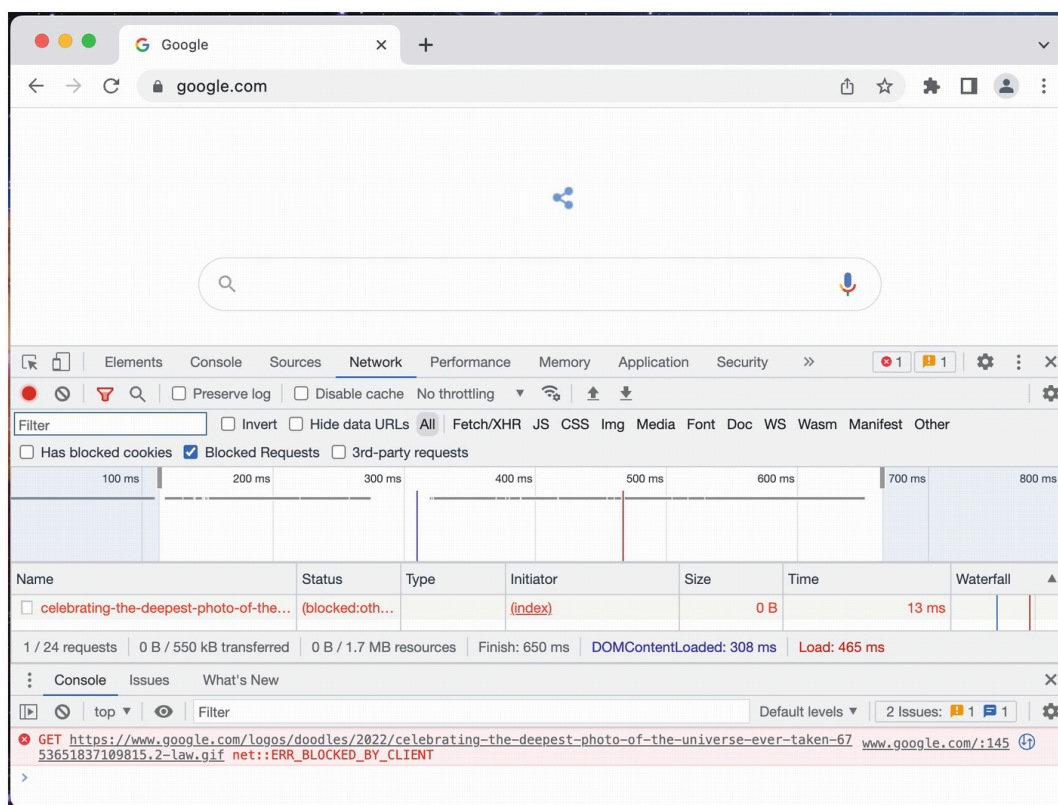


Figure 6-3 google.com with the request for the logo blocked

The DeclarativeNetRequest API imposes some problematic limitations on ad blockers. Google Chrome enforces a *global static rule limit*, meaning that all extensions installed in a browser each contribute toward a combined sum of rules. At the time that this book was written that total was 150,000, although the number may continue to fluctuate based on community feedback. This may seem to be a generous number – but considering that uBlock Origin alone defines on the order of 100,000 rules, this limit will quickly be reached.

Furthermore, the request matching capabilities of `declarativeNetRequest` are painfully limited when compared to what `webRequest` could offer: blocking requests based on payload size, disabling of JavaScript execution through the injection of CSP directives, and stripping cookie headers from outgoing requests. Because of these limita-

tions, some browsers like Firefox appear to be positioning themselves to preserve the features of `webRequest` in manifest v3.

Note For further reading on how manifest v3 affects ad blockers, refer to the following threads:

<https://github.com/uBlockOrigin/uBlock-issues/issues/338>

<https://bugs.chromium.org/p/chromium/issues/detail?id=896897>

Summary

The changeover from manifest v2 to manifest v3 has major implications for the browser extension ecosystem. The most dramatic changes are service workers and `declarativeNetRequest`, both of which have broad implications for how extensions behave. With these changes, extensions become more secure and performant, but at the same time it appears that some valuable features are being killed entirely.

The next chapter will explore background scripts. It will cover in depth how they work, how best to build them, and how they behave as the nerve center of browser extension.